III. AI분석모형

1. 최근접이웃(K-Nearest Neighbors)
2. 의사결정나무(Decision Tree)
3. 랜덤포레스트(Random Forest)
4. 앙상블(Ensemble)
5. 서포트벡터머신(SVM)
6. 군집분석(Clustering Analysis)

III. AI분석모형

1 최근접 이웃(K-Nearest Neighbors)

최근접 이웃

알고리즘은 우리가 예측하려고 하는 임의의 데이터와 가장 가까운 거리의 데이터 K개를 찾아 다수결에 의해 데이터를 예측하는 방법이다.

〈K-근접법 예시〉

새롭게 추가된 별 모양 데이터 주변에 Class B 데이터가 더 많으므로
별 모양 데이터는 Class B로 분류된다.

위 그림과 같이 두 그룹의 데이터가 있을 때 주어진 임의의 데이터가 어느 그룹에 속할 것인지를 K-NN알고리즘으로 풀 수 있다. 별 데이터로부터 가장 가까운 K개 데이터들이 더 많이 속한 그룹으로 별 모양 데이터를 분류하는 알고리즘이다. K가 3이라면 3개의 데이터를 확인해 더 많은 데이터가 있는 B그룹으로 주어진 임의의 데이터를 분류한다. K의 선택 기준은 학습의 난이도와 데이터의 개수이며, 보통 훈련 데이터 개수의 제곱근으로 설정한다.

K와 별 모양 데이터 간 좌표상 거리도 고려해야 한다. K를 4로 설정했다면 별 모양 데이터를 기준으로 4개의 데이터 간 떨어진 거리가 다르다. 별 모양 데이터를 기준으로 파란 데이터 중 1개 데이터의 거리가 가장 멀어 가중치가 가장 낮으므로 B그룹으로 분류된다.

K-NN의 장단점

K-NN알고리즘의 장점은 추가된 데이터의 처리가 쉽고, 예측 결과에 대한 해석도 쉽다는 것이다. 사용이 간단하여 훈련 데이터에 대한 훈련 과정이 별도로 필요하지 않으며, 범주를 나눈 기준을 알지 못하더라도 데이터를 분류할 수 있다.

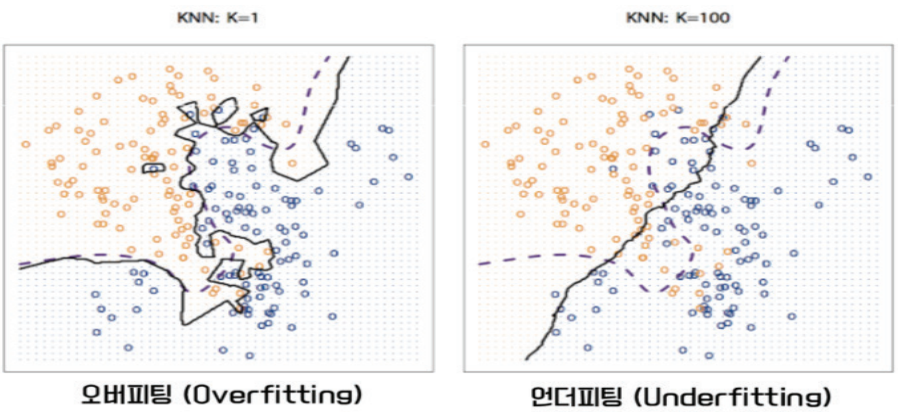
반면 훈련 데이터 세트의 크기가 너무 크거나 특성(feature)의 개수가 너무 많으면 연산속도가 느려지는 단점이 있다. 비수치 데이터의 유사도를 정의하기가 어렵고 이상치의 영향을 크게 받는다. 서로 다른 특성 값들의 비율을 일정하게 하지 않을 경우 성능이 떨어지기 때문에 같은 스케일(규모)을 갖도록 전처리 하는 과정이 필수적이다.

최근접 이웃을 활용한 회귀분석 사례

지도학습 방법으로 클래스가 정해진 데이터에 대해 분류 모델을 만들어 분류예측에 사용하거나 최근접 이웃 회귀분석에도 사용할 수 있다.

K값이 너무 작으면 거리가 가장 가까운 데이터 1개를 기준 삼아 새로운 데이터를 분류하게 되므로 분류 기준이 매우 엄격해진다. 훈련 데이터 세트에 대해서는 높은 정확도를 기대할 수 있지만 임의의 테스트 데이터 세트에는 높은 정확도를 기대하기 어려워진다. 결과적으로 과대적합(overfitting) 문제가 발생할 수 있다.

K값이 너무 크면 거리가 가장 가까운 K개의 모든 데이터를 기준으로 데이터를 분류하기 때문에 모델의 분류 기준이 매우 모호해진다. K가 극단적으로 커질 경우 훈련 데이터에 대해서 정확도 성능이 나오지 않는 과소적합(underfitting) 문제가 생길 수 있다.



〈K값 1과 100 차이〉

K값에 따른 차이점을 명확히 보여준다.

따라서 최근접 이웃 알고리즘을 사용할 때는 적절한 K의 개수를 정하는 것이 매우 중요하다. 일반적으로 K-NN 분류 모델을 훈련시킬 때 일정 범주를 주고 학습을 시키며, 교차검증이나 리샘플링 기법을 이용하여 최적의 성능을 내는 K 값을 구한다.

분석연습

K-NN알고리즘을 이용한 당뇨병 예측

DATASET

National Institute of Diabetes and Digestive and Kidney Diseases에서 제공된 당뇨병 진단 장치로부터 나온 데이터와 당뇨병의 여부를 보여준다. 샘플의 개수는 총 768개이고 21세 이상의 여성이다. 데이터 특성(feature)은 임신횟수, BMI, 인슐린 Level, 나이 등이 있으며, 결과는 당뇨병의 여부(0, 1)로 표현되어 있다.

1) 데이터 확인하기

```
df = pd.read_csv('./diabetes.csv')
df.head()
```

```
> df.head()
```

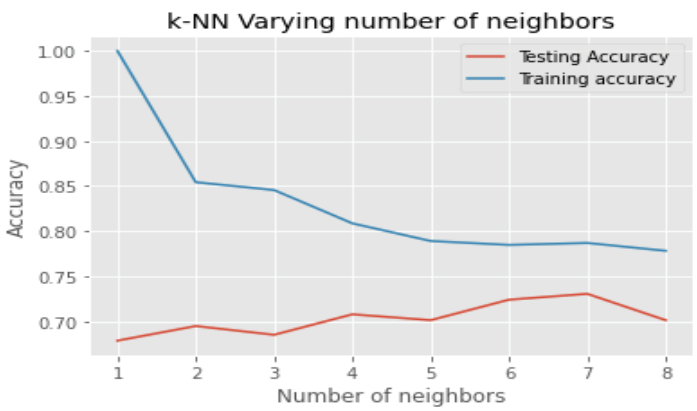
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

2) 훈련, 시험 데이터 분할 (6:4)

```
X_train,X_test,y_train,y_test = train_test_split(X_rescale,y,test_size=0.4,random_state=42, stratify=y)
```

3) K값을 변경하면서 알고리즘 정확도 측정

```
neighbors = np.arange(1,9)
train_accuracy =np.empty(len(neighbors))
test_accuracy = np.empty(len(neighbors))

for i,k in enumerate(neighbors):
    #Setup a knn classifier with k neighbors
    knn = KNeighborsClassifier(n_neighbors=k)

    #Fit the model
    knn.fit(X_train, y_train)

    #Compute accuracy on the training set
    train_accuracy[i] = knn.score(X_train, y_train)

    #Compute accuracy on the test set
    test_accuracy[i] = knn.score(X_test, y_test)
```



〈K값을 변경하며 정확도 측정을 했을 때 7에서 가장 차이가 적게 나타나고 있다.〉

4) 훈련 데이터와 시험 데이터 정확도의 차이가 가장 작은 K=7인 값을 이용하여 정확도 측정

```
confusion_matrix(y_test,y_pred)
pd.crosstab(y_test, y_pred, rownames=['실제'], colnames=['KNN모델'], margins=True)
```

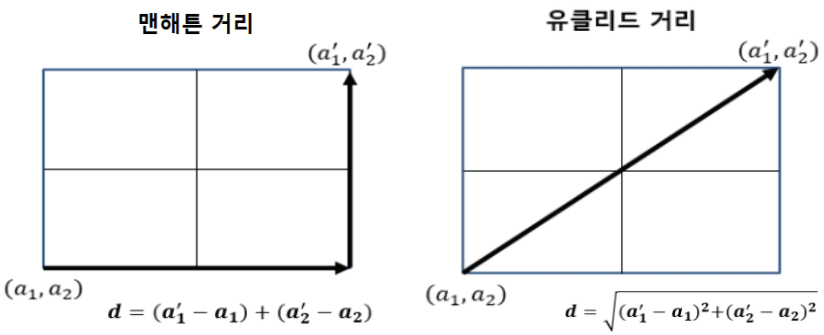
KNN모델	0	1	All
실제			
0	161	40	201
1	40	67	107
All	201	107	308

	precision	recall	f1-score	support
0	0.78	0.82	0.80	201
1	0.62	0.56	0.59	107
accuracy			0.73	308
macro avg	0.70	0.69	0.70	308
weighted avg	0.73	0.73	0.73	308

민코프스키 공식을 이용한 K-NN의 거리측정

```
knn = KNeighborsClassifier(n_neighbors=k, p=2, metric='minkowski')
```

최근접 이웃 알고리즘에서 사용되는 데이터 간 거리를 구하는 방식에는 '맨해튼'과 '유클리드' 측정 방식이 있다. 맨해튼 거리 측정 방식은 두 점 사이의 격자를 고려한 거리 측정 방식이고, 유클리드 거리 측정 방식은 최단거리를 측정한 것이다. 다음은 맨해튼과 유클리드 거리를 나타낸 그림과 공식이다.



$$d = \sqrt[p]{\sum_k (a'_k - a_k)^p}$$

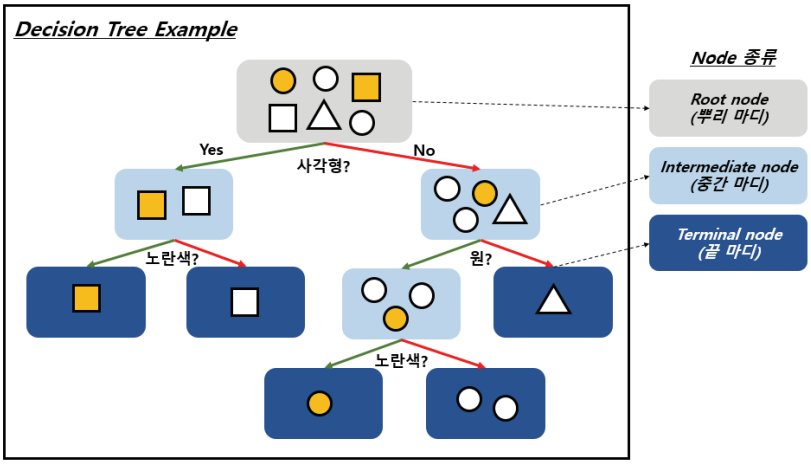
〈민코프스키 공식〉

민코프스키 공식은 거리를 구하는 공식을 일반화해 놓은 것이며, 위 식과 같다. 이때 P=1이면 맨해튼 거리, P=2일 때 유클리드 거리를 계산한다. 파이썬의 KNeighborsClassifier 함수에서 인자값을 생략 할 경우, 민코프스키에 P=2(유클리드 거리)가 기본값으로 설정되어 있다.

2 의사결정나무(Decision Tree)

의사결정나무

일종의 분류기법이다. 전체 집단을 계속 양분하는 분류기법으로써 분기가 발생하는 포인트(=노드)에는 기준이 되는 질문이 있어 기준 질문에 부합하냐(YES), 부합하지 않느냐(NO)에 따라 노드 이동의 방향이 결정된다.



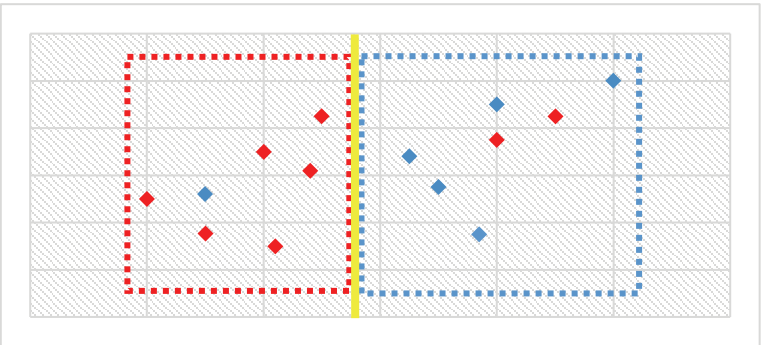
〈의사결정나무 예시〉
그림출처 : Wiki

의사결정나무 모형은 분류(classification)와 회귀예측(regression) 모두 가능한 알고리즘이다. 분류나무 모형은 불연속적(이산형 자료)인 값을 예측한다. 예를 들어 분류 모델은 다음과 같은 질문에 대한 답을 예측한다. '수신된 이메일이 스팸인가 아닌가?', '이 사진이 강아지인가 고양이인가 또는 햄스터인가?'와 같은 질문에 답을 내기 적합한 비지도 분류 알고리즘 모형이다.

한편 회귀분석의 한 갈래인 회귀나무 모형은 연속적인 값을 예측한다. 예를 들어 회귀 모델은 다음과 같은 질문에 답을 예측한다. '원주시 주택 가격은 얼마인가?', '사용자가 이 광고를 클릭할 확률이 얼마인가?'와 같은 질문에 답할 수 있다. 주로 의사결정 나무는 광고 인쇄물의 응답자 분석, 고객들의 신용점수화, 의학 연구, 시장분석, 품질관리, 주가, 환율 예측 등 다양한 분야에서 이용된다.

분할규칙

데이터 집단을 나눌 경우에는 분할 기준이 있다. 분할 기준은 분할 변수와 목표 변수를 통해 산포된 데이터들을 가장 잘 구분해 줄 수 있는 지점(기준)을 찾는 기준이 된다. 아래 그림에서처럼 분할 기준을 찾아가는 방법이 바로 분류나무이며, 이를 통해 나눠진 데이터 그룹은 그 특징을 보다 정확하게 설명할 수 있게 한다. 붉은 영역과 푸른 영역은 각각 다른 데이터를 포함하지만, 다수의 동일 데이터로 구성되어 있어 각 그룹을 구분 짓는 것이 가능하다.



〈데이터 분할 예시〉
노랑색 선을 기준으로 데이터를 구분하면 데이터의 성격을
이전보다 더 명확하게 이해할 수 있다.

순수도/불순도(Purity/Impurity)

위 그림처럼 데이터 분할이 이뤄졌을 때, 붉은 영역이 푸른 영역보다 순수도가 더 높다 또는 불순도(=혼잡도)가 더 낮다고 말할 수 있다. 분할점은 순수도가 최대(=불순도가 최소)가 되도록 설정된다. 불순도가 낮으면 불확실성도 감소하는데 이를 정보이론에서는 정보획득(Information Gain)이라고 한다.

불순도와 엔트로피(Entropy)

결정트리에서 불순도를 측정하는 지표로 엔트로피가 적용되어 있다. 물리학에서 엔트로피가 높다는 것은 무질서가 강하다는 의미다. 정보이론에서도 엔트로피가 높을수록 정보 내용의 기대수준이 떨어지는 현상을 정리했다. 데이터 혼잡도가 높을 경우 엔트로피값도 높게 나타난다.

결정트리분류기(DecisionTreeClassifier)

결정트리분류기(DecisionTreeClassifier)를 통해 아이리스 정보의 몇 가지 속성 가운데 잎사귀 길이(length)와 너비(width)를 기준으로 분류하는 과정을 이해할 수 있다. 아래는 잎사귀 크기 범위에 포함되느냐 아니냐에 따라 데이터를 분할하는 과정을 반복하는 중간중간의 결과들을 그래프로 하나씩 살펴본다.

참고 SOURCECODE

```
from sklearn.datasets import load_iris
data = load_iris()
y = data.target
X = data.data[:, 2:]

feature_names = data.feature_names[2:]

from sklearn.tree import DecisionTreeClassifier
tree1 = DecisionTreeClassifier(criterion='entropy', max_depth=1, random_state=0).fit(X, y)

import io
#import pydot
from IPython.core.display import Image
from sklearn.tree import export_graphviz

def draw_decision_tree(model):
    dot_buf = io.StringIO()
    export_graphviz(model, out_file=dot_buf, feature_names=feature_names)
    graph = pydot.graph_from_dot_data(dot_buf.getvalue())[0]
    image = graph.create_png()
    return Image(image)

print(Image)

def plot_decision_regions(X, y, model, title):
    resolution = 0.01
    markers = ('s', '^', 'o')
    colors = ('red', 'blue', 'lightgreen')
    cmap = mpl.colors.ListedColormap(colors)
    cmap

    x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, resolution),
                           np.arange(x2_min, x2_max, resolution))
    Z = model.predict(
        np.array([xx1.ravel(), xx2.ravel()]).T).reshape(xx1.shape)

    plt.contour(xx1, xx2, Z, cmap=mpl.colors.ListedColormap(['k']))
    plt.contourf(xx1, xx2, Z, alpha=0.4, cmap=cmap)
    plt.xlim(xx1.min(), xx1.max())
    plt.ylim(xx2.min(), xx2.max())

    for idx, cl in enumerate(np.unique(y)):
        plt.scatter(x=X[y == cl, 0], y=X[y == cl, 1], alpha=0.8,
                    c=[cmap(idx)], marker=markers[idx], s=80, label=cl)

    plt.xlabel(data.feature_names[2])
    plt.ylabel(data.feature_names[3])
    plt.legend(loc='upper left')
    plt.title(title)

    return Z
```

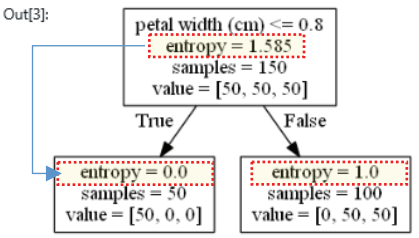
〈결정트리 분할 과정 예시〉

트리 함수가 반환한 결과를 Contour 형태로 시각화하여
몇 번의 분할을 어떻게 거쳤는지 확인한다.

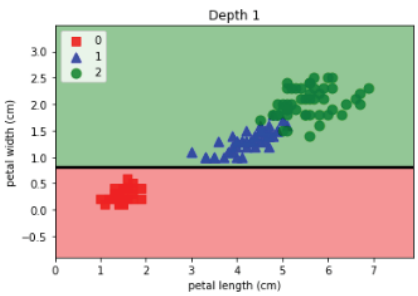
1) 먼저 잎사귀 길이(x)와 너비(y)값의 산포에서 붉은색 영역의 데이터 집단이 매우 독립적인 것을 확인할 수 있다. 여기서 가장 최적의 분류는 잎 너비 변수의 값이 0.8cm일 때이다.

```
In [3]: draw_decision_tree(tree1)
```

Out[3]:




```
In [4]: plot_decision_regions(X, y, tree1, "Depth 1")
        plt.show()
```



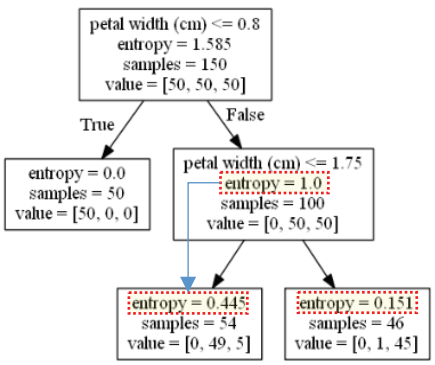
〈첫 번째 데이터 분할〉

데이터를 1:2 크기로 나눈 상황에서
엔트로피가 0인 클래스를 확인할 수 있다.

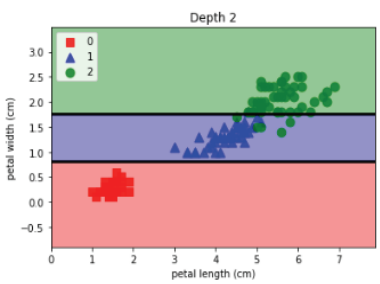
2-1) 다음은 엔트로피가 1인 데이터 그룹을 분할하는 과정이다. 데이터를 비슷한 규모로 양분한 최적의 분할 기준값은 잎사귀 너비 1.75cm이다.

```
In [7]: draw_decision_tree(tree2)
```

Out[7]:




```
In [8]: plot_decision_regions(X, y, tree2, "Depth 2")
        plt.show()
```



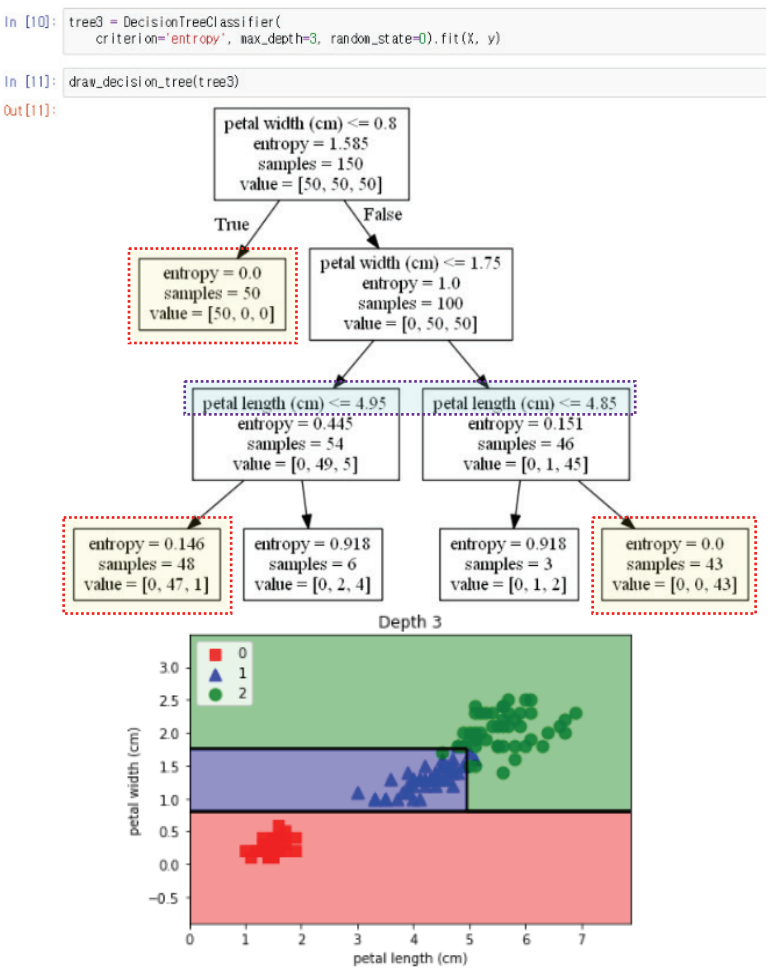
〈두 번째 데이터 분할〉

최적의 분할값을 찾아 분류하여도
엔트로피가 0이 아닐 수 있다.

2-2) 데이터를 비슷한 규모로 양분한 기준을 찾았음에도 파란색 영역에 녹색점이 5건 섞여 있다. 불순도가 1/10을 넘으므로 분할을 한 차례 추가한다.

3-1) 분할 기준을 다른 축에서 찾을 수도 있다. 이번엔 잎의 길이가 기준이 되어 4.95cm 이하와 4.85cm 이상을 나눈 결과를 확인한다. 결과적으로 잎 너비 1.75cm 이하, 길이 4.85cm 이하의 영역이 파란색으로 분류됐다.

3-2) 녹색 영역에 파란 점 1건이 존재하긴 하나 이를 제거하기 위해 트리가 한 차례 더 분류를 진행하는 건 무의미하다. 이미 잎 길이 4.95cm 분할 기준을 높이게 되면 파란색 영역의 불순도가 더 커지게 되기 때문이다(여러 분할 시뮬레이션 가운데 엔트로피 값이 가장 낮을 때(엔트로피 총합: 0.146) 분할 과정은 종료된다).



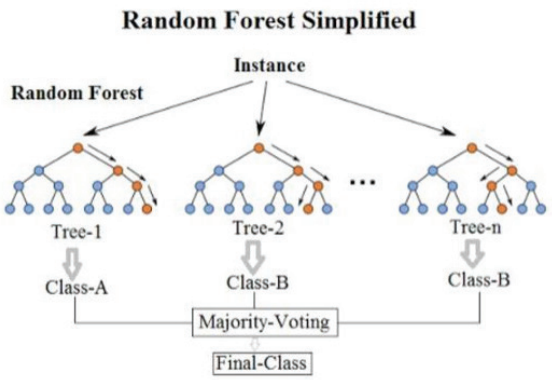
〈세 번째 분할〉

엔트로피가 0이 되는 또 다른 기준을 찾은 결과는
잎 너비 1.75 이하, 길이 4.85 이하의 범위(파란색 영역)이다.

❸ 랜덤포레스트(Random Forest)

랜덤포레스트(RF)

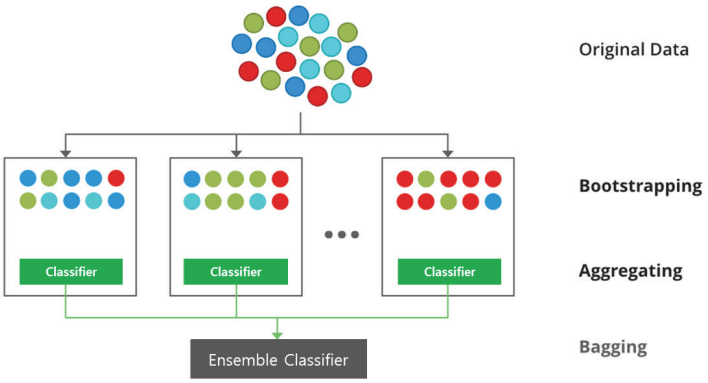
의사결정나무를 여러개 모아서 데이터 분류 및 예측을 수행하는 AI알고리즘이다. 어떤 데이터 집단에 대한 분류나 예측을 실시한다고 할 때, 하나의 결정트리를 사용하는 것보다 여러 트리를 결합해서 사용하면 보다 높은 성능의 알고리즘 모형을 만들 수 있다.



〈랜덤포레스트 구성 예시〉
여러 결정트리를 이용하여 답을 구한다. 하나의 트리만을 이용해
답을 찾는 방식보다 더 높은 성능을 기대할 수 있다.
그림출처 : Wiki

여러 나무와 하나의 포레스트, 배깅(Bagging, Bootstrap Aggregating)

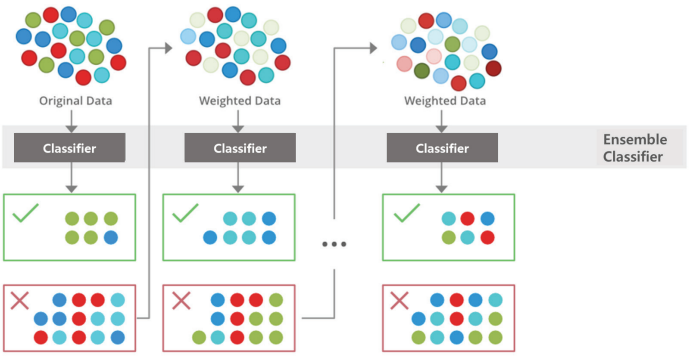
결정트리를 독립병렬 결합하는 방식을 배깅이라고 한다. 원 자료에서 하위 데이터 세트를 만드는 부트스트랩 과정을 포함한다. 데이터 건수는 동일하게 그리고 데이터 무작위 추출 시 중복은 허용(복원추출)하여 각 트리 모형들이 학습할 데이터 세트를 그 수에 맞게 준비해야 한다. 가령 결정트리 서른 개를 사용한다면, 전체 자료에서 데이터를 추출하여 30개의 학습 데이터 세트를 구성해야 한다.



트리마다 독립적으로 데이터 세트를 학습하고 결과를 낸다.
그림출처 : GeeksforGeeks

여러 나무와 하나의 포레스트, 부스팅(Boosting)

결정트리를 연결병렬 결합하는 방식을 부스팅이라고 한다. 첫 번째 결정트리의 결과가 그 다음 결정트리의 학습 데이터 구성에 영향을 미치는 순환구조이다. 무작위 샘플 복원추출과 여러 트리의 답을 최종 결정 시 다수결의 원칙에 맞춘다는 점은 배깅과 동일하다.



〈부스팅 과정 예시〉
이전에 트리의 결과 정보가 반영된 데이터 세트를 학습하고 결과를 낸다.
그림출처 : GeeksforGeeks

두 포레스트 유형 간 주요 차이 4가지

No	Bagging	Boosting
1.	동일 형태의 예측 조합	이종 형태의 예측 조합
2.	분산 감소 목적, 편향 그대로	편향 감소 목적, 분산 그대로
3.	모든 트리별 동일 가중치	트리별 성능 기반 차별적 가중치
4.	트리별 독립적인 학습 데이터 세트 구성	이전 트리에서 잘못 분류된 데이터를 다음 트리 학습 데이터 세트 구성에 포함

〈배깅과 부스팅의 주요 차이점〉
배깅은 병렬적인 결합으로 트리 모두 독립적으로 운용되며,
부스팅은 직렬적인 결합으로 트리별 전후 관계가 중요하다.

하나의 포레스트와 하나의 답, 다수결 원칙(Majority-Voting)

결정트리가 여러 개라면 각각의 결과 역시 여러 개일 것이다. 여러 결과를 모았을 때, 그 대표 결과를 선정하는 것은 다수결 원칙에 따른다. 예를 들어, 결정트리 3개 중 2개의 결과가 '사자'라면 나머지 하나의 트리에서 '고양이'라는 결과가 나오더라도 최종의 결과는 '사자'가 된다.

랜덤포레스트의 분류모형(RandomForestClassifier)

랜덤포레스트의 분류모형(RandomForestClassifier)을 활용하여 아이리스(iris)의 종 분류 시 RFC분석 과정을 알아본다. 파이썬 연습에 대표 예제인 아아리스 데이터 세트를 사용하여 모델이 스스로 아이리스의 종류를 나눈 결과를 대칭행렬(4 x 4)로 시각화하여 확인한다.

데이터 세트에는 아이리스의 속성 정보가 기록돼 있다. 잎사귀의 너비, 꽃잎의 색상 및 크기 등 총 4가지 주요 정보를 바탕으로 모델은 해당 속성 정보를 분석해 특징을 도출하고 그 특징을 기준으로 어떤 데이터가 어떤 종류의 아이리스인지를 분류하게 될 것이다.

참고 SOURCECODE

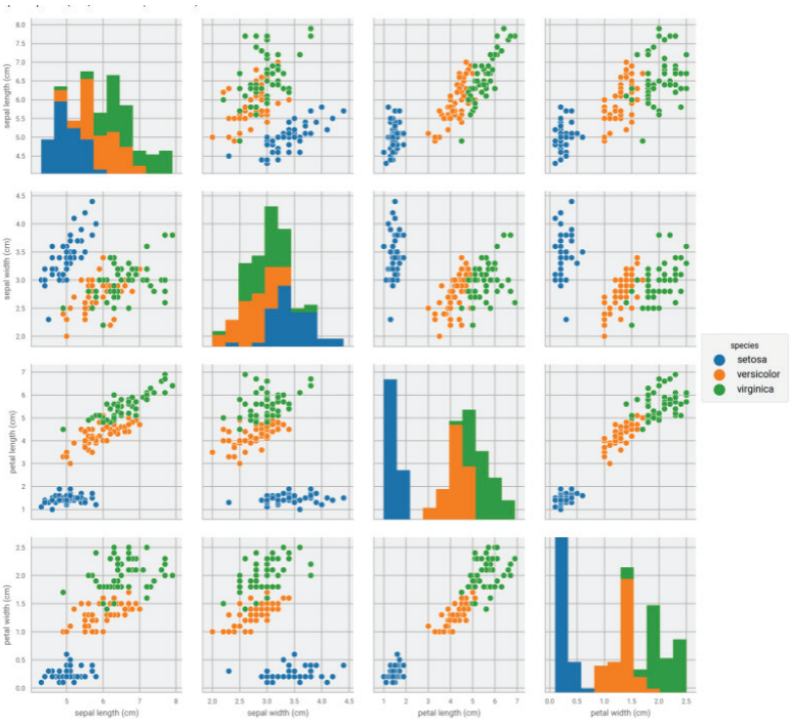
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

from sklearn import datasets

iris = datasets.load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)
#print(df)
# sklearn provides the iris species as integer values since this is required for classification
# here we're just adding a column with the species names to the dataframe for visualisation
df['species'] = np.array([iris.target_names[i] for i in iris.target]) ---
#print(df['species'])

sns.pairplot(df, hue='species')
# ------------------------------------------------------------------ 데이터 프레임으로 변경

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(df[iris.feature_names], iris.target, test_size=0.25, s
# ------------------------------------------------------------------ 학습/훈련데이터 구분
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, oob_score=True, random_state=9999, bootstrap=True)
rf.fit(X_train, y_train)

from sklearn.metrics import accuracy_score

predicted = rf.predict(X_test)
accuracy = accuracy_score(y_test, predicted)

print(f'Out-of-bag score estimate: {rf.oob_score_:.3}')
print(f'Mean accuracy score: {accuracy:.3}')

from sklearn.metrics import confusion_matrix

cm = pd.DataFrame(confusion_matrix(y_test, predicted), columns=iris.target_names, index=iris.target_names)
sns.heatmap(cm, annot=True)
```

〈랜덤포레스트 분류모형 소스코드 예시〉

scikit-learn 라이브러리의 랜덤포레스트 분류기를 통해
아이리스 속성 정보를 가지고 꽃의 종류를 나눈다.

의사결정나무에서 확인했던 모델이 아이리스 데이터를 언제까지 분할하는지를 상기한다. 데이터의 불순도가 낮아지는(엔트로피가 0에 가까운) 상태까지 모델이 데이터 분할을 시도할 것이다.

산포도를 중심으로 보면 파란색 데이터가 가장 먼저 분류가 이뤄졌을 것이란 예상이 가능하다. 변수 간 관계와 정밀도(=모여있는 정도)가 제일 명확하기 때문이다. 문제는 데이터 중첩이 많은 주황색과 녹색 자료인데, 불순도가 최소가 되는 지점에서 여러 결정트리의 분류 결과를 계산(다수결)해 최종적으로 두 자료를 나누게 된다. 결과적으로 총 3종으로 아이리스 데이터가 표현되었다.



〈아이리스 속성 4가지의 시각화 대칭행렬〉

속성별(4×4) 데이터 분포로 모델은 총 3종으로 아이리스 데이터 세트를 분류하였다.

scikit-learn 라이브러리에는 랜덤포레스트의 분류모형과 회귀모형 모두 있다. 랜덤포레스트 알고리즘은 이산형 자료 바탕의 '분류'와 연속형 자료 바탕의 '예측'을 목적으로 한다.

랜덤포레스트의 회귀모형(RandomForestRegressor)

랜덤포레스트의 회귀모형(RandomForestRegressor)을 활용하여 와인 맛 평가에 가장 중요한 요인이 무엇인지 확인한다. 여기서는 와인 성분이 정리된 정보표를 RFR모델에 학습시키고 해당 모델의 어떤 성분이 와인 종류를 나누는 데 가장 큰 영향을 미치는 성분인지를 예측하는 과정을 확인한다.

SOURCECODE와 설명

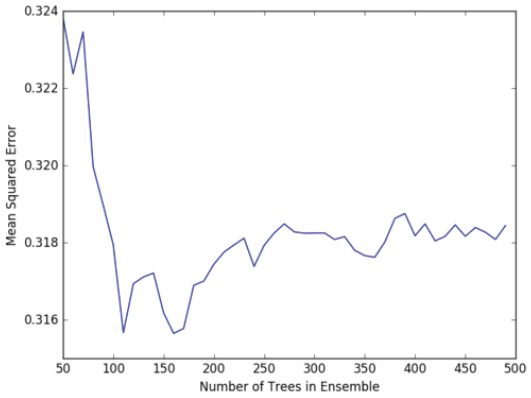
```
nrows = len(xList)
ncols = len(xList[0])
X = numpy.array(xList)
Y = numpy.array(labels)
wineNames = numpy.array(names)
#print(X);print(Y);print(wineNames)

#데이터 행의 30%로 고정된 홀드 아웃 세트 구성
xTrain, xTest, yTrain, yTest = train_test_split(X, Y, test_size = 0.3, random_state = 531
#print(xTrain);print(xTest);print(yTrain);print(yTest)

#MSE의 변화를 확인하기 위하여 앙상블의 크기 범위에서 랜덤포레스트 트레이닝
mseOos = []
nTreeList = range(50, 500, 10)
for iTrees in nTreeList:
    depth = None
    maxFeat = 4 #조정해 볼 것
    wineRFModel = ensemble.RandomForestRegressor(n_estimators = iTrees,
                        max_depth = depth, max_features = maxFeat,
                        oob_score = False, random_state = 531)
    wineRFModel.fit(xTrain, yTrain)

    #데이터 세트에 대한 MSE 누적
    prediction = wineRFModel.predict(xTest)
    mseOos.append(mean_squared_error(yTest, prediction))

print("MSE")
print(mseOos[-1])

#트레이닝 테스트 오차 대비 앙상블의 트리 개수 도표 그리기
plot.plot(nTreeList, mseOos)
plot.xlabel('Number of Trees in Ensemble')
plot.ylabel('Mean Squared Error')
#plot.ylim([0.0, 1.1*max(mseOob)])
plot.show()
```

〈랜덤포레스트 회귀모형 소스코드 예시〉

scikit-learn 라이브러리의 랜덤포레스트 회귀모형을 활용하여
적정한 트리 개수와 와인 맛 구성에 중요한 성분 값을 찾는다.

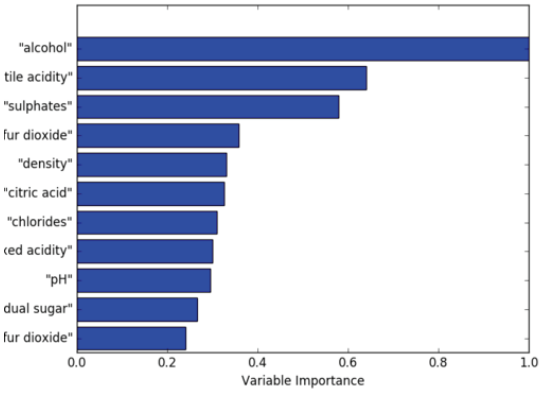
랜덤포레스트(Random Forest) : 실습

랜덤포레스트에서 결정트리의 수가 정확도를 항상 보장하진 않는다. 아래 그림에서 처럼 알고리즘은 결정트리의 적정한 수와 더불어 원 자료를 근거로 마련된 데이터 학습이 잘 선행되어야 일정 수준의 성능을 기대할 수 있다.



〈결정트리의 개수와 오류 발생의 관계 검정〉
결정트리도 최적의 수가 있다. 오분류율이 가장 낮은 점이 최적이다.

와인 종류 정보를 포함해 와인을 구성하는 성분표 데이터 세트를 **RF**회귀 모델에 학습시키고 해당 모델에서 어떤 성분이 와인 맛(=종류)을 결정하는 데 유의미하게 영향을 미쳤는지를 예측한 결과이다. 그래프를 통해 확인하면 알코올(alcohol)과 산도(tile acidity) 등의 성분이 와인 맛을 가르는 중요한 성분으로 예측됐다.

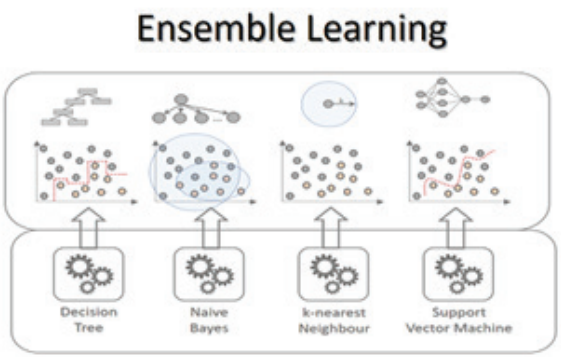


〈와인 맛 평가 주요 요인 분석 결과〉
와인을 구성하는 여러 성분 요소 가운데 알코올이
와인의 종류를 나누는 가장 강력한 성분임을 보여준다.

④ 앙상블(Ensemble)

앙상블 학습(Ensemble Learning)

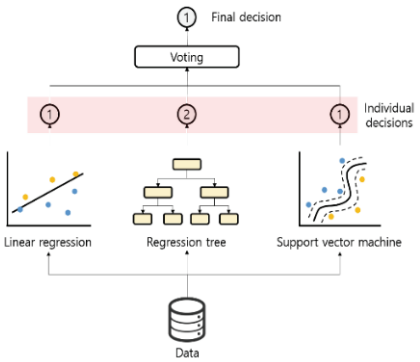
그동안 우리는 여러 기계학습 모델들을 살펴보았다. 이렇게 개별적으로 동작하는 모델들을 모아 종합적으로 의사결정을 한다면 어떨까? 앙상블은 프랑스어로 전체적인 어울림이나 통일을 의미하며 음악에서 2인 이상의 가창이나 합주를 의미하기도 한다. 예측력을 높이기 위해 여러 가지 학습알고리즘이 어울려 성능을 높이는 방식을 기계학습 분야에서 앙상블(Ensemble) 학습이라 한다.



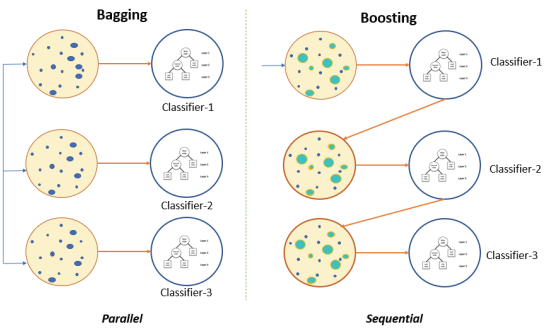
〈앙상블 학습〉

앙상블 종류

앙상블 학습 방식에는 크게 Voting , Bagging, Boosting의 3가지로 나눌 수 있다. Voting(투표) 방식은 여러 모델로부터 나온 후보 결과들 중 다수가 나온 결과를 채택하는 방식이다. Bagging은 같은 모델을 사용하지만 훈련세트에서 중복을 허용한 여러 부분 집합을 샘플링하여 학습하는 방식이며, 여기서 모델이 의사결정나무를 이용한 것을 랜덤포레스트라고 한다. 마지막으로 Boosting은 약한 개별의 모델들을 연결하여 이전 모델이 생성한 가중치 또는 오차를 기준으로 성능을 보완해 나가면서 학습하는 방법이다.



〈Voting〉

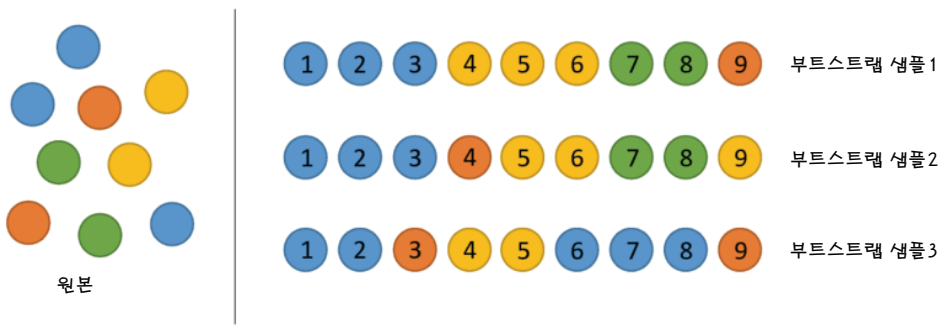


〈Bagging과 Boosting〉

앙상블의 장단점

일반적으로 앙상블 학습 방법을 사용하면, 개별 모델에서 나온 결과를 종합하는 과정에서 종합 결괏값인 분산이 줄어들게 된다. 즉 특정 훈련 데이터 세트에서만 정확도를 보장하는 과대적합(Overfitting)현상을 어느 정도 줄여 줄 수 있다. 다만 투표방식인 Voting의 경우 앙상블에 포함된 모델 사이의 독립성이 전제되는 경우 개별 분류기보다 정확한 예측이 가능하지만 그렇지 못한 경우 성능이 떨어질 수 있다.

배깅(Bagging) vs 페이스팅(Pasting)



배깅은 Bootstrap Aggregating의 줄임말로 중복을 허용한 무작위 표본 추출에 의존하는 어떤 시험이나 계측으로 훈련세트를 구성하고, 페이스팅은 중복을 허용하지 않고 데이터 세트를 구성하는 것을 말한다.

예제

단일 모델과 앙상블 기법(Voting, Bagging, Boosting)을 이용한 유방암 분류 예측 비교 분석

DATASET

유방암 조직에서 미세 바늘을 이용하여 세포를 획득한 후 이미지를 디지털화한 정보이다. 특성은 이미지 속 수치 정보 10개(둘레, 반지름, 질감 등)를 각각 평균, 표준편차, 가장 큰 3개 값의 평균으로 하여 총 30개이며 결과는 악성(Malignant), 양성(benign)이다.

1) 데이터 확인하기

```
cancer = load_breast_cancer()

df = pd.DataFrame(cancer.data, columns=cancer.feature_names)

df.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	184.60	2019.0
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.41	158.80	1956.0
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.53	152.50	1709.0
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91	26.50	98.87	567.7
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	22.54	16.67	152.20	1575.0

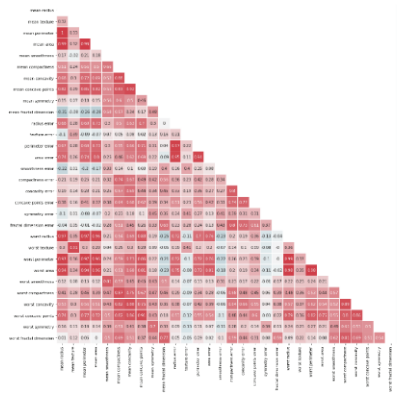
5 rows × 30 columns

2) 탐색적 데이터 분석 – 결측치 확인 및 공분산 매트릭스를 통한 변수 제거

```
# Generate and visualize the correlation matrix
corr = df.corr().round(2)

# Mask for the upper triangle
mask = np.zeros_like(corr, dtype=np.bool)

mask[np.triu_indices_from(mask)] = True

# Set figure size
f, ax = plt.subplots(figsize=(20, 20))

# Define custom colormap
cmap = sns.diverging_palette(220, 10, as_cmap=True)

# Draw the heatmap
sns.heatmap(corr, mask=mask, cmap=cmap, vmin=-1, vmax=1, center=0,
            square=True, linewidths=.5, cbar_kws={"shrink": .5}, annot=True)
```

가장 큰 3개 값의 평균, 둘레 넓이, Concavity, Concave point 데이터 간의 강한 상관관계가 발견되어 특성값 제거 (30개 ▶ 12개)

```
cols = ['worst radius', 'worst texture', 'worst perimeter', 'worst area',
        'worst smoothness', 'worst compactness','worst concavity',
        'worst concave points', 'worst symmetry', 'worst fractal dimension']
df = df.drop(cols, axis=1)

cols = ['mean perimeter', 'perimeter error', 'mean area','area error']
df = df.drop(cols, axis=1)
```



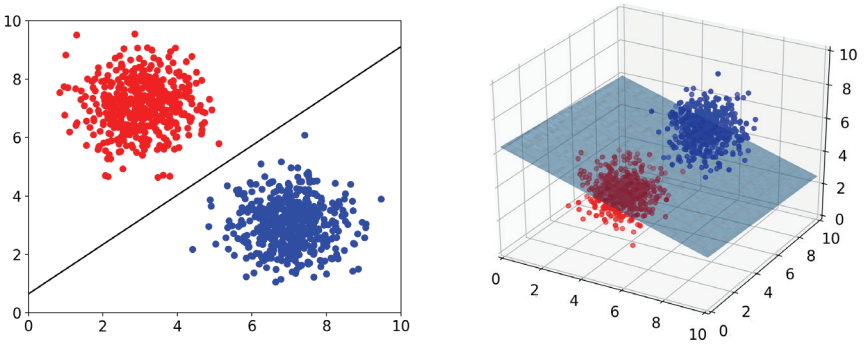
〈특성 30개〉

〈특성 12개〉

5 서포트벡터머신(SVM)

서포트벡터머신(SVM)

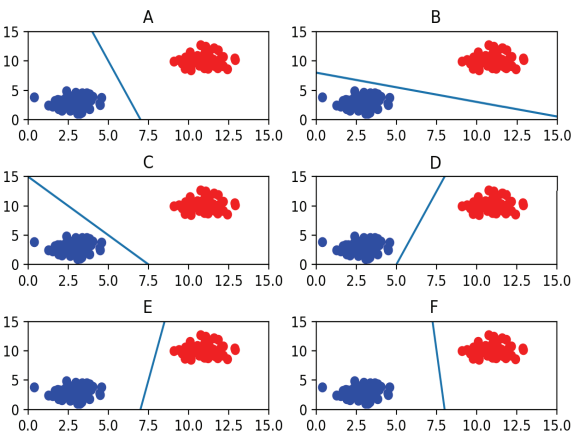
분류를 위한 기준선을 정의하는 모델이다. 분류되지 않은 새로운 점이 나타나면 어느 쪽에 속하는지 확인을 위해서 결정 경계를 정한다.



〈결정 경계(Decision Boundary) 이해〉
2차원과 3차원에서 두 그룹을 분류하는 최적의 경계를 찾는다.
(이하 내용 및 그림출처 : ojune575 블로그)

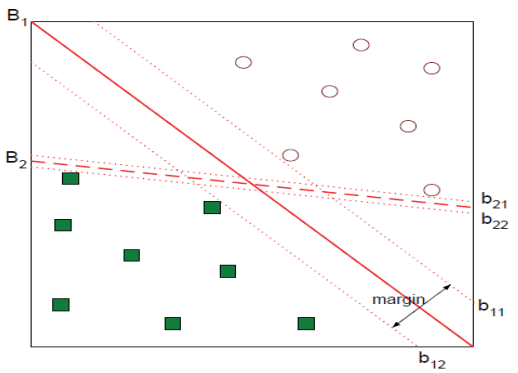
데이터에 2개 속성(feature)만 있다면 결정 경계는 간단한 선 형태가 된다. 속성이 3개가 되면 3차원으로 그려야 한다. 이때의 결정 경계는 '선'이 아닌 '평면'이 된다. 우리가 이렇게 시각적으로 인지할 수 있는 범위는 딱 3차원까지지다. 차원, 즉 속성의 개수가 늘어날수록 당연히 복잡해질 것이다. 결정 경계도 단순한 평면이 아닌 고차원이 되는데 이를 '초평면(hyperplane)'이라고 부른다.

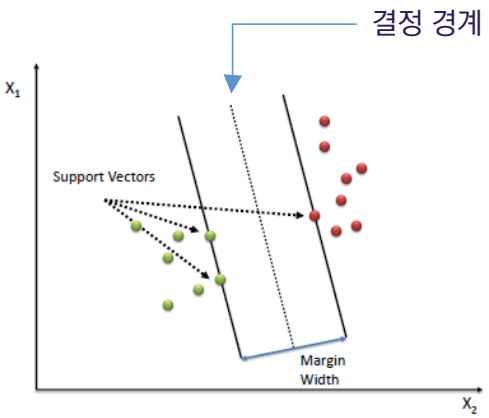
결정 경계는 여러 형태로 정할 수 있다. 그래프 C를 보면 경계선이 파란색 클래스(분류)와 너무 가까워 보인다. 결정 경계가 가장 적절한 것은 그래프 F다. 두 클래스 사이에서 거리가 가장 멀기 때문이다. 결정 경계는 데이터 군으로부터 최대한 멀리 떨어지도록 설정하는 것이 좋다.



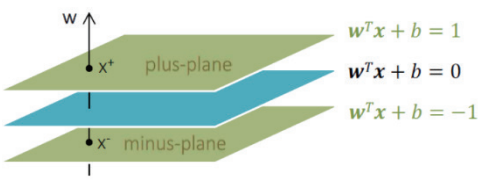
마진(Margin)의 이해

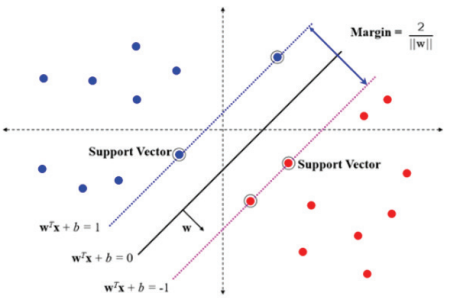
마진(Margin)은 결정 경계와 서포트 벡터 사이의 거리를 의미한다.





위의 왼쪽 그림에서 직선 B1와 B2 모두 두 클래스를 무난하게 분류하고 있음을 확인할 수 있다. 좀 더 나은 결정경계를 꼽으라면 B1일 것이다. 두 범주를 여유 있게 구분하고 있다. b11을 plus-plane, b12를 minus-plane, 이 둘 사이의 거리가 마진(margin)이다. 결정 경계에 가장 가까운 각 클래스의 점들(데이터 포인트들)을 서포트벡터(support vertors)라 한다. 마진의 결정에 영향을 끼치는 관측치들로써, 이 데이터들이 경계를 정의하는 결정적인 역할을 하는 셈이다. SVM은 이 마진을 최대화하는 분류 경계면을 찾는 기법이라 할 수 있다.

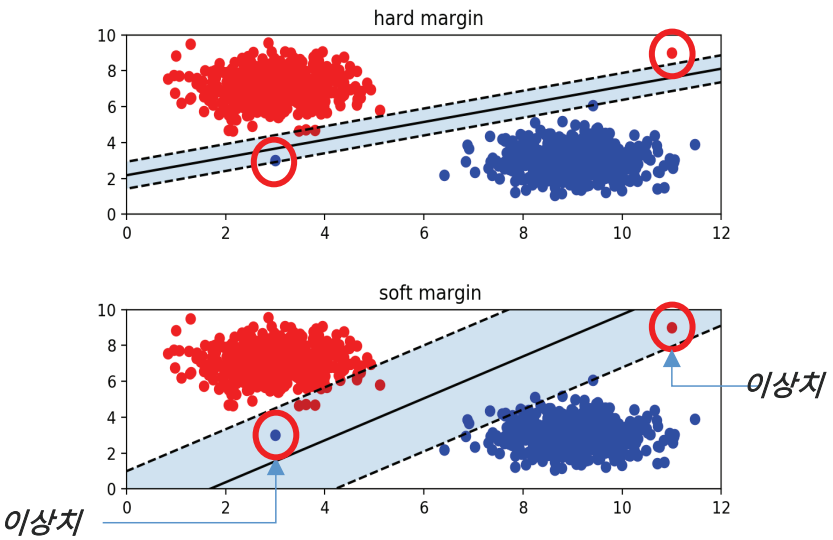




경계면을 도식적으로 나타내면 위의 왼쪽 그림과 같다. 도식화된 분류 경계면을 $w^T + b$ 이라고 할 때 벡터 w는 이 경계면과 수직인 법선벡터가 되며, 오른쪽 그림과 같이 마진의 폭은 $\frac{2}{||w||}$ 으로 정의된다. 마진의 폭이 최대가 되려면 벡터 **w**가 최소가 되어야 한다.

이상치(극단값, outlier)

이상치는 '패턴에서 벗어난 값' 또는 '중심에서 많이 벗어난 값'을 의미한다. 통계적 자료 분석의 결과를 왜곡시키거나 자료 분석의 적절성을 위협하는 변숫값을 뜻하기도 한다.



왼쪽에 혼자 떨어져 있는 파란 점과, 오른쪽에 혼자 떨어져 있는 빨간 점이 이상치이다.

위쪽 그림은 서포트 벡터와 결정 경계 사이의 거리가 매우 좁아져서 마진이 매우 작아진 상태이다. 이런 상태를 하드 마진(hard margin)이라고 한다. 이상치를 허용하지 않는 기준으로 결정 경계를 정해버리면 과대적합(overfitting) 문제가 발생할 수 있다.

아래쪽 그림은 이상치들이 마진 안에 어느 정도 포함되도록 기준을 잡았다. 서포트 벡터와 결정 경계 사이의 거리가 멀어져서 마진이 커진 상태이다. 이런 상태를 소프트 마진(soft margin)이라고 한다. 이 경우 과소적합(underfitting) 문제가 발생할 수 있다.

문제정의

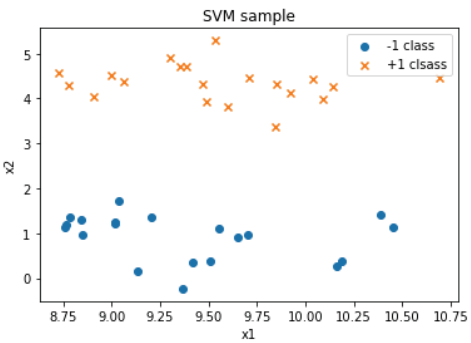
분리된 두 클래스를 선형으로 가장 적절하게 결정 경계를 정하도록 하자.

DATASET

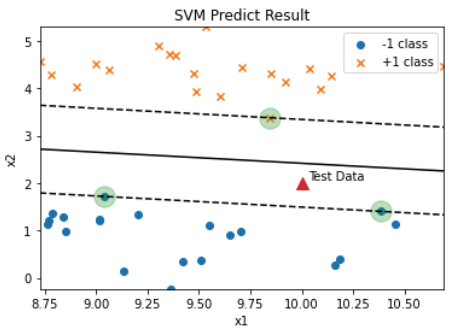
데이터 등 관련 자료와 설명

각 클래스를 구분하기 위해 'O'와 'X'로 데이터를 표시하고, 'O'는 '-1 class'로 'X'는 '+1 class'로 구분하여 표시한다. 샘플데이터는 각 클래스당 20개로 설정한다.

```
from sklearn.datasets import make_blobs
X, y = make_blobs(n_samples=40, centers=2,
        cluster_std=0.5, random_state=4)
y = 2 * y - 1
plt.scatter(X[y == -1, 0], X[y == -1, 1],
  marker='o', label="-1 class")
plt.scatter(X[y == +1, 0], X[y == +1, 1],
  marker='x', label="+1 class")
plt.xlabel("x1")
plt.ylabel("x2")
plt.legend()
plt.title("SVM sample")
plt.show()
```




```
import numpy as np
from sklearn.svm import SVC
model = SVC(kernel='linear', C=1e10).fit(X, y)
xmin = X[:, 0].min()
xmax = X[:, 0].max()
ymin = X[:, 1].min()
ymax = X[:, 1].max()
xx = np.linspace(xmin, xmax, 10)
yy = np.linspace(ymin, ymax, 10)
...
plt.scatter(X[y == +1, 0], X[y == +1, 1],
marker='x', label="+1 class")
plt.contour(X1, X2, Z, levels, colors='k',
linestyles=linestyles)
plt.scatter(model.support_vectors_[:, 0],
model.support_vectors_[:, 1], s=300, alpha=0.3)
x_new = [10, 2]
plt.scatter(x_new[0], x_new[1], marker='^',
s=100)
plt.text(x_new[0] + 0.03, x_new[1] + 0.08, "Test
Data")
plt.xlabel("x1")
plt.ylabel("x2")
plt.legend()
plt.title("SVM Predict Result")
plt.show()
```



〈SVM 예측 결과〉

Scikit-Learn의 SVM 패키지는 서포트 벡터 머신 모형인 SVC(Support Vector Classifier) 클래스를 제공한다.

문제정의

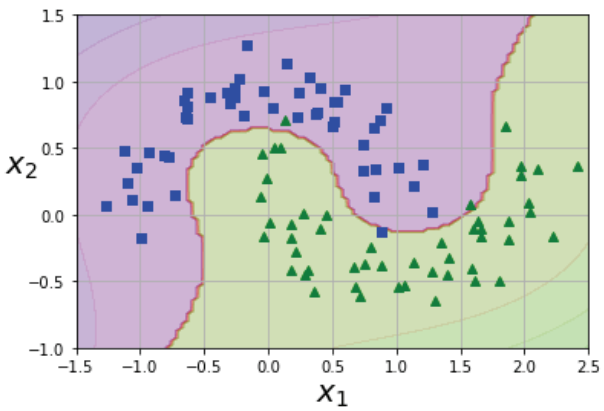
분리된 두 클래스가 비선형인 경우 결정 경계를 정하도록 하자.

DATASET

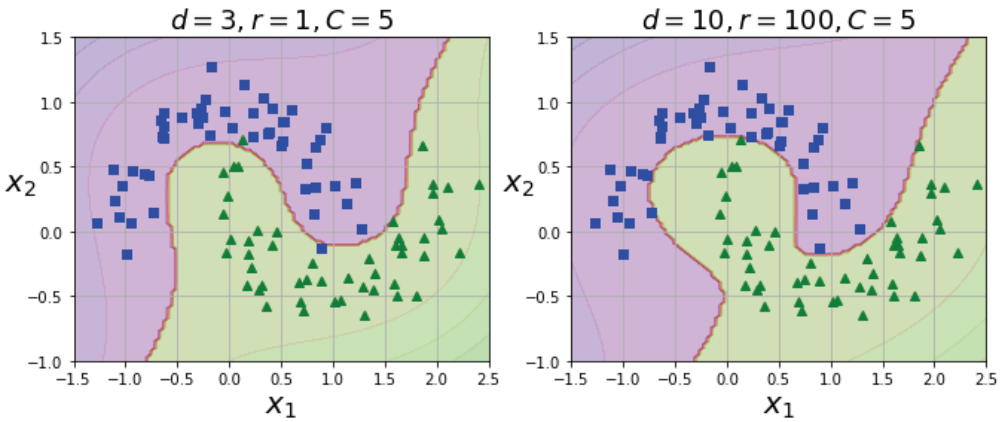
데이터 등 관련 자료와 설명

하나의 클래스는 '□' , 다른 클래스는 '△'로 구분하여 표시한다. 커널은 다항식(poly)을 사용한다.

```
import numpy as np
from matplotlib import pyplot as plt
def plot_dataset(X, y, axes):
    plt.plot(X[:, 0][y==0], X[:, 1][y==0], "bs")
    plt.plot(X[:, 0][y==1], X[:, 1][y==1], "g^")
    plt.axis(axes)
    plt.grid(True, which='both')
    plt.xlabel(r"$x_1$", fontsize=20)
    plt.ylabel(r"$x_2$", fontsize=20, rotation=0)
def plot_predictions(clf, axes):
    x0s = np.linspace(axes[0], axes[1], 100)#axes[0]부터 axes[1]까지 100개로 이루어진 숫자들
    x1s = np.linspace(axes[2], axes[3], 100)
    x0, x1 = np.meshgrid(x0s, x1s) #표현할 수 있는 모든 배열조합
    X = np.c_[x0.ravel(), x1.ravel()]# ravel 1차원 배열로 핀다.
    y_pred = clf.predict(X).reshape(x0.shape)
    y_decision = clf.decision_function(X).reshape(x0.shape)
    plt.contourf(x0, x1, y_pred, cmap=plt.cm.brg, alpha=0.2)
    plt.contourf(x0, x1, y_decision, cmap=plt.cm.brg, alpha=0.1)

# 다항식(polynomial) 커널을 사용하면 데이터를 더 높은 차원으로 변형하여 나타냄으로써 초
평면(hyperplane)의 결정 경계를 얻을 수 있다.
plot_predictions(polynomial_svm_clf, [-1.5, 2.5, -1, 1.5])

plot_dataset(X, Y, [-1.5, 2.5, -1, 1.5])
plt.show()
```



```
from sklearn.svm import SVC
poly_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
])
#coef0는 모델이 높은 차수와 낮은 차수에 얼마나 영향을 끼치는지 정할 수 있다.
poly_kernel_svm_clf.fit(X, Y)

plt.figure(figsize=(11, 4))
plt.subplot(121)
plot_predictions(poly_kernel_svm_clf, [-1.5, 2.5, -1, 1.5])
plot_dataset(X, Y, [-1.5, 2.5, -1, 1.5])
plt.title(r"$d=3, r=1, C=5$", fontsize=18)

poly_kernel_svm100_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="poly", degree=10, coef0=100, C=5))
])
poly_kernel_svm100_clf.fit(X, Y)

plt.subplot(122)
plot_predictions(poly_kernel_svm100_clf, [-1.5, 2.5, -1, 1.5])
plot_dataset(X, Y, [-1.5, 2.5, -1, 1.5])
plt.title(r"$d=10, r=100, C=5$", fontsize=18)
plt.show()
```

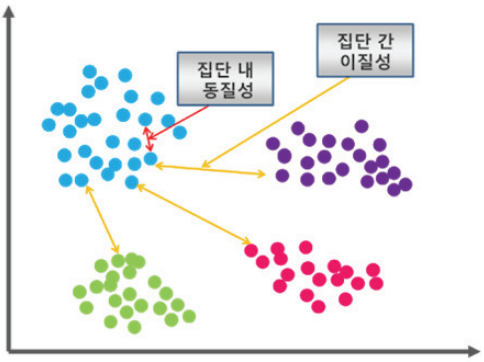


매개변수인 coef( ) 값을 1에서 100으로 변경할 경우 결정 경계가 오른쪽 그림과 같이 달라짐을 알 수 있다.

6 군집분석(Clustering Analysis)

군집분석

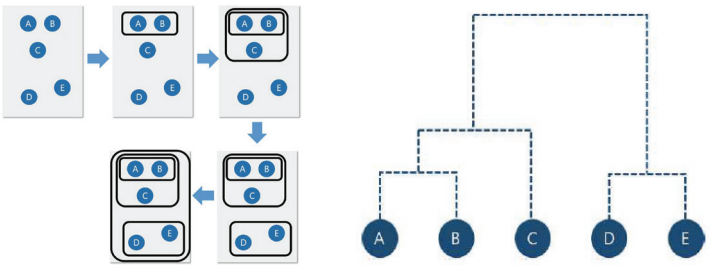
분류 기준이 없는 상태에서 데이터 속성을 고려해 스스로 전체 데이터를 N개의 소그룹으로 묶어내는(Clustering) 분석법이다. 유사성이 높은 대상 데이터를 묶고, 서로 다른 그룹에 속한 데이터 개체 간의 이질성을 계산하는 과정을 거친다.



〈군집분석 예시〉
군집화는 집단 내 동질성과 이질성을 기반으로 이뤄진다.

계층적 군집화(Hierarchical Clustering)

가장 유사한 개체를 묶어 나가는 과정을 반복하여 원하는 개수의 군집을 형성하는 방법이다. 계층적 군집을 형성하는 방법에는 작은 군집으로부터 출발하여 군집을 병합해 나가는 병합적 방법과 큰 군집으로부터 출발하여 군집을 분리해 나가는 분할적 방법이 있다.



왼쪽 그림은 병합적 방법으로 군집을 형성해 나가는 과정을 보여주고 있다.
계층적 군집의 결과는 오른쪽 그림과 같이 덴드로그램(dendrogram)의 형태로 표현된다.
이 그림을 통해 군집들 간의 구조적 관계를 파악할 수 있다. 항목 간의 거리, 군집 간의 거리를 알 수 있고, 군집 내의 항목 간 유사 정도를 파악함으로써 군집의 견고성을 해석할 수 있다.

계층적 군집 종류

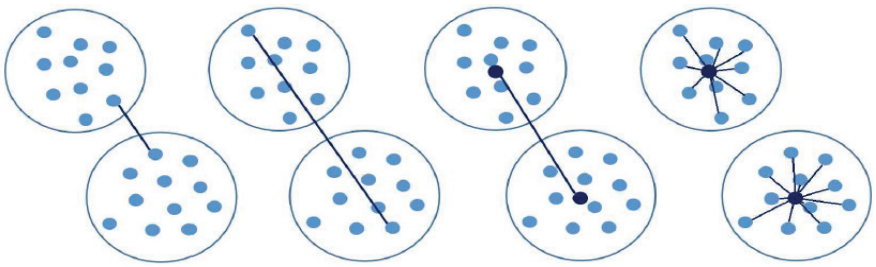
1) 최단 연결법(single linkage, nearest neighbor) :
한 군집의 점과 다른 군집의 점 사이에 가장 짧은 거리로 군집 형성

2) 최장 연결법(complete linkage, farthest neighbor) :
한 군집의 점과 다른 군집의 점 사이에 가장 긴 거리로 군집 형성

3) 중심 연결법(centroid linkage) : 두 군집의 중심 간의 거리로 군집 형성

4) 평균 연결법(average linkage) : 한 군집의 점과 다른 군집의 점 사이의 거리에 대한 평균을 사용하여 군집 형성

5) 와드 연결법(ward linkage) :
군집 간 거리에 기반하는 다른 연결법과는 달리, 군집 내 오차제곱합(ESS, error sum of square)을 고려하여 군집 형성(군집 간 정보의 손실을 최소화하기 위해 군집화)



〈최단 연결법〉 〈최장 연결법〉 〈중심 연결법〉 〈와드 연결법〉

비계층적 군집(Non-Hierarchical Clustering Method)

구하고자 하는 군집의 수를 정한 상태에서 설정된 군집의 중심에 가장 가까운 개체를 하나씩 포함해 가는 군집 형성 방법이다.

비계층적 군집 종류

1) K-평균 군집(K-means clustering) : 사전에 결정된 군집 수 K에 기초하여 전체 데이터를 상대적으로 유사한 K개의 군집으로 구분하는 방법이다. 가장 대표적인 방법이며, 순차적 군집 분석법(Sequential Threshold Method)이라고도 한다.

2) 동시 군집 분석법(Paralleled Threshold Method) : 사전에 지정된 값 안에 관측 대상이나 속성이 속하는 경우나 몇 개의 군집이 동시에 결정되는 경우

3) 최적 할당 군집 분석법(Optimizing Partitioning Method) : 사전에 주어진 군집의 수를 위한 군집 내 평균거리를 계산하는 최적화 기준에 의거하여 최초의 군집에서 다른 군집으로 다시 할당하는 방법

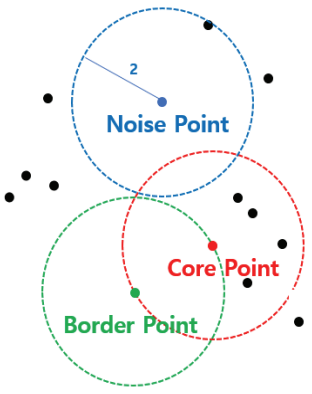
K-평균 군집(K-means clustering) 과정

1) 클러스터 개수 K값 결정
2) 데이터 공간에 클러스터 중심 K개 할당
3) 각 클러스터 중심을 해당 클러스터 데이터의 평균으로 조정
4) 클러스터 중심이 변하지 않을 때까지 반복

DBSCAN 클러스터링

DBSCAN의 경우 '밀도 기반'으로 군집을 할당한다. 여기서 밀도 기반이란, 어떤 데이터 포인트에서의 반지름 x내에 n개 이상의 포인트를 갖는 것을 하나의 군집으로 구분하는 것이다. 즉 K-Means와는 달리 군집의 개수 k를 미리 정의해놓을 필요는 없지만 반지름과 한 군집 내에 최소 n개의 포인트가 있어야 하므로 이 두 개를 사전에 정의해야 한다.

오른쪽 그림에서 반지름이 2이고, 최소 6개의 포인트를 가질 때 1개의 군집으로 할당하고자 한다. 이때 빨간 점은 해당 조건을 만족하여 코어 포인트가 된다. 보더 포인트란 코어 포인트의 반지름 내에 걸쳐서 '이웃'하고 있으나 최소 포인트 개수를 충족시키지 못하는 경우(초록색)를 말한다. 노이즈 포인트는 코어 포인트도 아니고 보더 포인트도 아닌 포인트를 말한다. 모든 데이터 포인트에서 해당 점이 코어 포인트가 되는지 확인하면서 군집을 할당하는 것이 DBSCAN 알고리즘의 핵심이다. K-Means와 달리 어느 군집에도 속하지 않는 노이즈 포인트는 제외하며 남아있는 포인트를 중심으로 군집을 할당하는 것이 특징이다. 노이즈 포인트를 버리기 때문에 노이즈에 취약하지 않으며, 군집의 모양과 사이즈가 다양하게 나올 수 있다.



DBSCAN에서 최적의 반지름과 최소 군집 개수 찾는 법

이제 최적의 반지름과 최소 군집 개수를 찾아보자. 이상적인 케이스를 생각하면, 하나의 군집 안에 있는 데이터 포인트의 거리는 가깝고, 노이즈 포인트와의 거리는 꽤 멀게 떨어져 있을 것이다. 어떤 군집 안에 있는 데이터 포인트들과 'k번째 인접한 이웃 데이터 포인트까지의 거리'를 계산해보며 반지름을 찾아나간다. 즉, 그 거리가 급격하게 늘어날 때까지의 점을 찾는다. 여기서 k는 최소 군집 개수이며, k번째 인접한 이웃 데이터 포인트까지의 거리는 반지름이 된다.

문제정의

계층적 군집 방법에 대해 알아보자.

DATASET

붓꽃 꽃받침 길이(Sepal Length), 꽃받침 폭(Sepal Width), 꽃잎 길이(Petal Length), 꽃잎 폭(Petal Width)

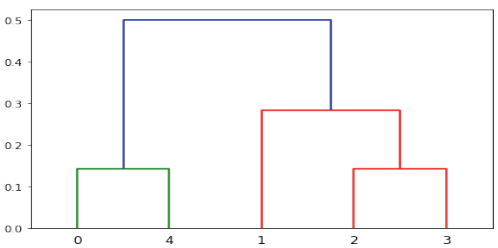
참고 SOURCECODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
#계층적 군집화
iris = load_iris()
#print(iris)
iris_df = pd.DataFrame(iris.data, columns =
iris.feature_names)
print(iris_df.head(5))
from scipy.spatial.distance import pdist, squareform
distmatrix = pdist(iris_df.loc[0:4, ["sepal length (cm)",
"sepal width (cm)"]],
                   metric = "euclidean")
#print("distmatrix: ", distmatrix)
#squareform을 이용해 거리 값을 matrix로 형태로 표현
row_dist = pd.DataFrame(squareform(distmatrix))
print("row_dist: ", row_dist)
```

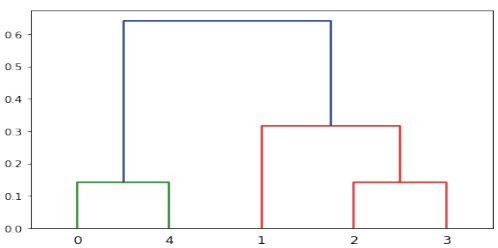
	Sepal length (cm)	Sepal width (cm)	Petal length (cm)	Petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

	0	1	2	3	4
0	0.000000	0.538516	0.500000	0.640312	0.141421
1	0.538516	0.000000	0.282843	0.316228	0.608276
2	0.500000	0.282843	0.000000	0.141421	0.500000
3	0.640312	0.316228	0.141421	0.000000	0.640312
4	0.141421	0.608276	0.500000	0.640312	0.000000

```
#단일(최단) 연결법
from scipy.cluster.hierarchy import linkage,
dendrogram
r_cluster = linkage(distmatrix, method = "single")
print("r_cluster: ", r_cluster)
df = pd.DataFrame(r_cluster, columns = ["id_1",
"id_2", "거리", "멤버 수"])
#print(df)
row_dend = dendrogram(r_cluster)
plt.tight_layout()
plt.show()
```




```
# 완전(최장) 연결법
from scipy.cluster.hierarchy import linkage,
dendrogram
r_cluster = linkage(distmatrix, method = "
complete")
print("r_cluster: ", r_cluster)
df = pd.DataFrame(r_cluster, columns = ["id_1",
"id_2", "거리", "멤버 수"])
#print(df)
row_dend = dendrogram(r_cluster)
plt.tight_layout()
plt.show()
```



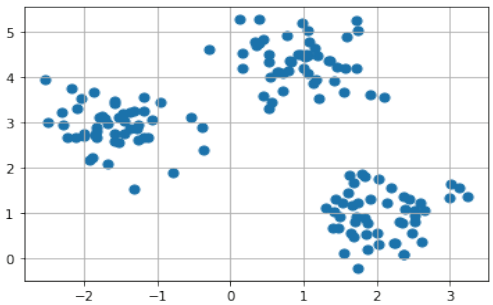
X축의 특성은 동일하게 군집하지만,
단일 연결법과 완전 연결법은 각각 Y축이 0.5일 때와
0.6일 때 연결되는 차이를 보임

문제정의

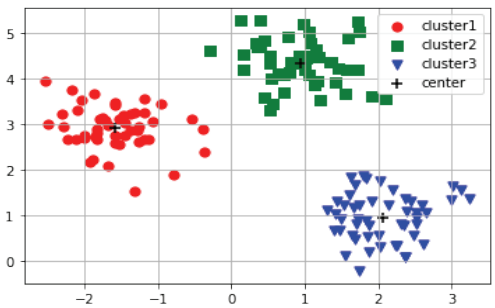
비계층적 군집 가운데 K-평균 군집(K-means Clustering)에 대해 알아보자.

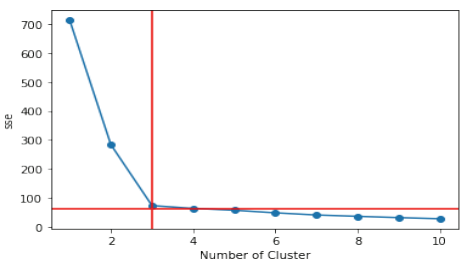
참고 SOURCECODE

```
#초기 Data 시각화
from sklearn.datasets import make_blobs
print(make_blobs)
x, y = make_blobs(n_samples = 150, n_features = 2,
centers = 3,
                  cluster_std = 0.5, shuffle = True,
random_state = 0)
print("x.shape: ", x.shape, ", y.shape: ", y.shape)

plt.scatter(x[:, 0], x[:,1], marker = "o", s = 50)
plt.grid()
plt.show()
```



```
#Kmeans Clustering
from sklearn.cluster import Kmeans
# 초기 클러스터 중심은 임의로 할당
init_centroid = "random"
#init_centroid = "k-means++" # 기본값
kmodel = KMeans(n_clusters = 3, init =
init_centroid, random_state = 0)
#print(kmodel)
pred = kmodel.fit_predict(x)
#print("pred: ", pred)
plt.scatter(x[pred == 0, 0], x[pred == 0, 1], marker
= "o", s = 50, c = "red", label = "cluster1")
plt.scatter(x[pred == 1, 0], x[pred == 1, 1], marker
= "s", s = 50, c = "green", label = "cluster2")
plt.scatter(x[pred == 2, 0], x[pred == 2, 1], marker
= "v", s = 50, c = "blue", label = "cluster3")
plt.scatter(kmodel.cluster_centers_[:,0],
kmodel.cluster_centers_[:,1],
            marker = "+", s = 80, c = "black", label =
"center")
plt.legend()
plt.grid()
plt.show()
```






```
#n-clusters를 알기위한 방법
#1 엘보우 기법: 클러스터 내 오차제곱합이 최소
가 되도록 클러스터의 중심을 결정해 나가는 방
법
def elbow(x):
  sse = []
  for i in range(1, 11):
    km = KMeans(n_clusters = i,
            init = "k-means++", random_state =
0)
    km.fit(x)
        sse.append(km.inertia_)
    print(sse[3])
    plt.plot(range(1, 11), sse, marker = "o")
    plt.axvline(x=3,color='r')
    plt.axhline(y=sse[3],color='r')
    plt.xlabel("Number of Cluster")
    plt.ylabel("sse")
    plt.show()
elbow(x)
# 클러스터의 개수가 3일 때 팔꿈치 부분이므로,
최적의 클러스터 개수는 3으로 결정됨
```

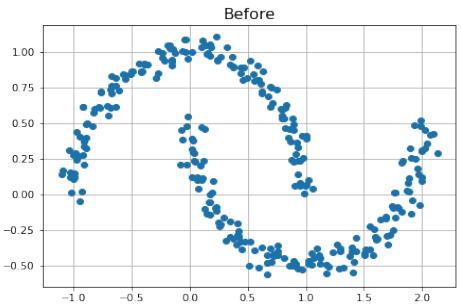
문제정의

계층적 군집 방법에 대해 알아보자.

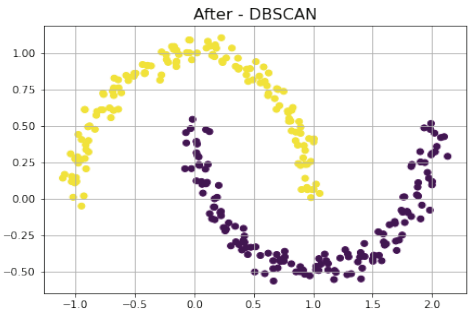
밀도 기반 군집화의 명확한 이해를 위해, 예제 데이터로 Moon 데이터 세트를 활용하였다. Moon 데이터는 make_moons 함수로 생성할 수 있으며, 샘플 수(n_samples)와 분산 정도(noise)를 조절해서 생성할 수 있다.

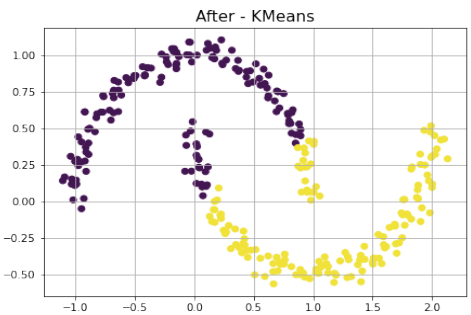
참고 SOURCECODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
x,y = make_moons(n_samples=300,
noise=0.05, random_state=42)
df=pd.DataFrame(x)
#df.head()
#산점도
plt.figure(figsize=(7,5))
plt.title("Before", fontsize=15)
plt.plot(df[0], df[1], "o")
plt.grid()
plt.show()
```




```
#DBSCAN함수로 클러스터링을 수행해보자
#eps : 반경 설정(속성값의 단위)
#min_samples : 최소 개체 수
from sklearn.cluster import DBSCAN
db_scan = DBSCAN(eps=0.3,
min_samples=5).fit(df.values)
df['cluster_db'] = db_scan.labels_
plt.figure(figsize=(7,5))
plt.title("After - DBSCAN", fontsize=15)
plt.scatter(df[0],df[1],c=df['cluster_db'])
plt.grid()
plt.show()
```




```
#K-means와 DBSCAN 비교
#위와 동일한 Moon 데이터 세트로 KMeans를
활용해 작업을 수행해본 결과이다.
from sklearn.cluster import KMeans
kmeans_ = KMeans(n_clusters=2,
random_state=42).fit(df.values)
df['cluster_km'] = kmeans_.labels_
plt.figure(figsize=(7,5))
plt.title("After - KMeans", fontsize=15)
plt.scatter(df[0],df[1],c=df['cluster_km'])
plt.grid()
plt.show()
```